The notation, (?:EXPRESSION) makes the subexpression inside the parenthesis into a group without generating backreferences. This may be useful when you need to group a set of constructs but don't want the group to set the value of $1 or whatever. In the example that follows, both patterns match a date with either colons or slashes between the month, day, and year. The first form stores the separator character (which can be a slash or a colon) in $2 and $4, but the second pattern uses (?: to avoid storing the separator in an external variable:

```
date = "12/25/2022"

date =~ %r{(\d+)(/|:)(\d+)(/|:)(\d+)}
[$1,$2,$3,$4,$5] # => ["12", "/", "25", "/", "2022"]

date =~ %r{(\d+)(?:/|:)(\d+)(?:/|:)(\d+)}
[$1,$2,$3]       # => ["12", "25", "2022"]
```

Using Dynamic Regular Expressions

You'll sometimes want to match a pattern only if the matched substring is preceded by or followed by some other pattern. That is, you want to set some context for your match but don't want to capture that context as part of the match.

For example, you might want to match every word in a string that is followed by a comma, but you don't want the comma to form part of the match. Here you could use the charmingly named *zero-width positive lookahead* extension. (?=EXPRESSION) matches EXPRESSION at this point but does not consume it—you can look forward for the context of a match without affecting the magic match variables like $&. In this example, we'll use String#scan to pick out the words that are followed by a comma:

```
str = "red, white, and blue"
str.scan(/[a-z]+(?=,)/) # => ["red", "white"]
```

You can also match before the pattern using (?<=EXPRESSION) (*zero-width positive lookbehind*). This lets you look for characters that precede the context of a match without affecting $&. The following example matches the letters *dog* but only if they are preceded by the letters *hot*:

```
show_regexp("seadog hotdog", /(?<=hot)dog/) # => seadog hot->dog<-
```

It's worth noting that while these complex regular expressions are definitely powerful, they can be very hard to read, and you can often get similar effects by combining simpler regular expressions with filtering code.

For the lookbehind extension, EXPRESSION either must be a fixed length or consist of a set of fixed-length alternatives. That is, (?<=aa) and (?<=aa|bbb) are valid, but (?&<=a+b) is not.

Both forms have negated versions, (?!EXPRESSION) and (?<!EXPRESSION), which are true if the context is not present in the target string.

The \K sequence is related to backtracking. If included in a pattern, it doesn't affect the matching process. However, when Ruby comes to store the entire matched string in $&, it only stores the text to the right of the \K:

```
show_regexp("thx1138", /[a-z]+\K\d+/) # => thx->1138<-
```